

Why V2 (NoBN) Explodes at Depth: A Note on the Per-Layer Variance Schedule

May 25, 2026

1 The two formulas are the same formula

The per-layer weight standard deviation is

$$\sigma_\ell = \sqrt{\frac{2}{d_{\ell-1}}} \cdot s^* \cdot r^{\eta(\ell-(L+1)/2)}. \quad (1)$$

We give the depth-dependent part its own name, s_ℓ :

$$s_\ell \equiv s^* \cdot r^{\eta(\ell-(L+1)/2)}. \quad (2)$$

Substituting (2) into (1) gives the compact form

$$\boxed{\sigma_\ell = \sqrt{\frac{2}{d_{\ell-1}}} \cdot s_\ell} \quad (3)$$

There is only one factor $r^{\eta(\cdots)}$. It lives inside s_ℓ . Equations (1) and (3) are identical; the second just abbreviates $s^* r^{\eta(\cdots)}$ as s_ℓ . No double counting.

The constants (for ReLU):

$$r = \sqrt{\frac{\pi-1}{\pi}} \approx 0.826 \quad (< 1), \quad s^* = \left(\frac{\pi}{\pi-1}\right)^{1/4} \approx 1.1006.$$

s^* is *always* 1.1006. It never changes. It is *not* 63.

2 What s_ℓ actually equals (where “63” comes from)

Take the run `cifar10/50L`, $\eta = 0.9$, and look at layer $\ell = 2$ ($L = 50$, hidden fan-in $d = 500$).

Step 1. Exponent:

$$\eta\left(\ell - \frac{L+1}{2}\right) = 0.9(2 - 25.5) = 0.9 \times (-23.5) = -21.15.$$

Step 2. Raise r to that exponent. Because $r < 1$, a *negative* power gives a *large* number:

$$r^{-21.15} = \left(\frac{1}{0.826}\right)^{21.15} = 1.21^{21.15} \approx 57.5.$$

Step 3. Multiply by the constant s^* to get s_ℓ :

$$s_2 = s^* \cdot r^{-21.15} = 1.10 \times 57.5 \approx \mathbf{63.3}.$$

This 63 is s_2 — not s^* . s^* is still 1.10.

Step 4. Multiply by the He factor to get the weight std:

$$\sigma_2 = \sqrt{\frac{2}{500}} \cdot s_2 = 0.0632 \times 63.3 \approx \mathbf{4.0}.$$

So s_ℓ is the constant $s^* = 1.10$ *after* the schedule multiplies it by the ramp $r^{\eta(\ell-(L+1)/2)}$. In the early layers that ramp is a big number (57 at layer 2), so s_ℓ becomes large (63), and the weight std becomes 4.0.

3 Why the $\sqrt{2/d}$ does *not* save us

You are right that $\sqrt{2/d_{\ell-1}}$ is the standard He compensation, and it **does** work: it exactly cancels the $\sqrt{d}$ gain of a matrix multiply.

The hidden $\sqrt{d}$. When a vector x (per-component RMS a) passes through a weight matrix $W \in \mathbb{R}^{d \times d}$ whose entries have std σ , each output is a sum of d products, so its RMS is

$$\text{rms}(Wx) \approx \sigma \sqrt{d} a.$$

The per-layer gain is therefore $\sigma \sqrt{d}$, *not* σ .

He cancels it. With $\sigma = \sqrt{2/d}$ we get $\sigma \sqrt{d} = \sqrt{2}$, and after ReLU (which removes half the energy) the net gain is ≈ 1 . Signal preserved at any depth.

But the schedule re-inserts s_ℓ . With the full σ_ℓ ,

$$\text{per-layer gain} = \sigma_\ell \sqrt{d} = \underbrace{\sqrt{\frac{2}{d}} \sqrt{d}}_{=\sqrt{2}} \cdot s_\ell = \sqrt{2} s_\ell.$$

After the ReLU/row-centering factor this becomes the clean net gain

$$\boxed{g_\ell = r s_\ell} \tag{4}$$

which **contains no d at all** — the $\sqrt{d}$ already cancelled. The gain is governed entirely by s_ℓ :

$$g_2 = r s_2 = 0.826 \times 63.3 \approx 52.$$

Each pass through layer 2 multiplies the signal by about 52.

The control case. If the schedule were uniform ($\eta = 0$), then $s_\ell = s^* = 1.10$ at every layer and

$$g_\ell = r s^* = 0.826 \times 1.10 = 0.909 \approx 1,$$

so nothing explodes — the $\sqrt{2/d}$ compensation handles everything. **The explosion is caused entirely by the η -schedule making s_ℓ large in the early layers; it is not a failure of the $\sqrt{2/d}$ term.**

4 How a std of ~ 4 explodes the activations

A single weight of value ~ 4 is harmless. What explodes is the *signal*, because the per-layer gains (4) *compound through depth*. For `cifar10/50L`, $\eta = 0.9$:

layer ℓ	s_ℓ	σ_ℓ	net gain $g_\ell = r s_\ell$
1	75.2	1.92	62
2	63.3	4.00	52
5	37.7	2.39	31
10	15.9	1.01	13
25	1.20	0.076	0.99
45	0.04	0.0024	0.03
50	0.02	0.0010	0.01

The early layers have gain $\gg 1$ (52, 31, 13, $\dots$); the late layers have gain $\ll 1$. The cumulative activation RMS is the running product of these gains. It **balloons** through the early high-gain layers, peaks near the middle, then is squashed back down by the late low-gain layers.

Why the network still “starts” at $L = 50$. The product of *all* 50 gains is ≈ 1 by design ($g_\ell = r s_\ell$, and the s_ℓ are symmetric about the middle), so input $\rightarrow$ output is balanced and the initial loss is a sane $\ln 10 \approx 2.30$. The activations bulge in the middle but stay finite, so the first batch computes. Then one SGD step perturbs the giant early weights, breaks the delicate up-then-down balance, and the activations blow up within a few steps (observed: NaN at batch 10).

Why $L = 100$ dies at birth. The bulge grows like $r^{-\eta L^2/8}$ — *quadratic in depth*. At $L = 100$ (even with a smaller $\eta = 0.36$, hence smaller individual s_ℓ) the running product reaches $\sim 10^{35}$ at the middle layer. Squared quantities inside the matmul then exceed the float32 ceiling (3.4×10^{38}), so the forward overflows to **inf** on the very first batch — the loss never even leaves 2.30 (observed: NaN at batch 8, loss frozen at 2.3026).

5 One-paragraph summary

The $\sqrt{2/d_{\ell-1}}$ term is intact and cancels the $\sqrt{d}$ of every matrix multiply, exactly as in He init. The constant $s^* = 1.10$ never changes. The explosion comes from the schedule factor $r^{\eta(\ell-(L+1)/2)}$: because $r < 1$, in the early layers (negative exponent) this becomes a large number (≈ 57 at layer 2), so $s_\ell = s^* r^{\eta(\dots)} \approx 63$ and the per-layer forward gain $g_\ell = r s_\ell \approx 52$. Those large early-layer gains compound through depth, ballooning the activation signal — mildly enough at $L = 50$ to start and then diverge under SGD, and severely enough at $L = 100$ to overflow float32 at initialization. Raising η (for a flatter gradient ratio) makes the early-layer s_ℓ larger, which is why better gradient conditioning and forward stability pull against each other — the Pareto trade-off.